ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 337E

Principles of Computer Communication
Assignment 1

GROUP MEMBERS:

150220939 : Madina Alzhanova

150220905 : Najib Attar

FALL 2025

Contents

FRONT COVER **1**

CONTENTS **1**

1 INTRODUCTION **1**

2 IMPLEMENTED ALGORITHMS **1**

2.1 Digital-to-Digital Conversion 1

2.1.1 Overview and Objective 1

2.1.2 Implemented Line Coding Schemes 1

2.1.3 NRZ-L Encoding 2

2.1.4 NRZ-I Encoding 2

2.1.5 Manchester Encoding 2

2.1.6 AMI Encoding 2

2.1.7 Implementation Structure 2

2.2 Digital-to-Analog Conversion 3

2.2.1 Overview and Objective 3

2.2.2 Implemented Modulation Schemes 3

2.2.3 Amplitude Shift Keying (ASK) 4

2.2.4 Frequency Shift Keying (FSK) 4

2.2.5 Binary Phase Shift Keying (BPSK) 4

2.2.6 Implementation Structure 4

2.2.7 Verification and Output 4

2.3 Analog-to-Digital Conversion 5

2.3.1 Overview and Objective 5

2.3.2 PCM Processing Stages 5

2.3.3 Analog Signal Model 6

2.3.4 Sampling 6

2.3.5 Quantization 6

2.3.6 PCM Encoding, Decoding, and Reconstruction 6

2.3.7 Implementation Structure and Verification 6

2.4 Analog-to-Analog Conversion 7

2.4.1 Overview and Objective 7

2.4.2 Implemented Modulation Schemes 7

2.4.3 Message and Carrier Signals 8

2.4.4 Amplitude Modulation (AM) 8

2.4.5 Phase Modulation (PM) 8

2.4.6 Frequency Modulation (FM) 8
2.4.7 Implementation Structure and Verification 8
2.5 Web Interface 9

3 AI OPTIMIZATION and BENCHMARK ANALYSIS 11
3.1 Claude's Optimization 11
3.2 Gemini's Optimization 14
3.3 Benchmark analysis 15

4 DISCUSSION 16
4.1 Digital-to-Digital Conversion 16
4.2 Digital-to-Analog Conversion 17
4.3 Analog-to-Digital Conversion 17
4.4 Analog-to-Analog Conversion 17
4.5 Overall Comparison 18
4.6 Difficulties encountered 18

5 Conclusion 18

REFERENCES 20

1 INTRODUCTION

In data communication systems, the nature of the transmitted signal is fundamental to the system's design. Signals are broadly categorized into two types: digital and analog. The transformation of data into these signal types is achieved through specific conversion processes known as encoding and modulation.

Digital signaling represents output data as discrete voltage pulses or levels. Analog signaling, the output data is continuous signal. Conversion of input signal (either analog or digital) to discrete signal is called encoding. Modulation is the process of encoding the input signal(small frequency) onto the carrier wave(high frequency). Depending on input and output signal nature, there are 4 ways to transmit data:

1. Digital-to-Digital
2. Digital-to-Analog
3. Analog-to-Digital
4. Analog-to-Analog

2 IMPLEMENTED ALGORITHMS

The project implements a modular framework that supports multiple signal conversion techniques. Each technique processes an input signal, performs the required conversion, and outputs a verified result.

2.1 Digital-to-Digital Conversion

2.1.1 Overview and Objective

Digital-to-digital conversion, or *line coding*, represents a digital bitstream as a digital signal suitable for transmission over a physical medium. The objective of this section is to implement and analyze common line coding schemes with respect to signal representation, synchronization capability, and DC content.

Several standard line coding techniques were implemented and visualized, each mapping binary data into specific voltage levels or transitions to enable correct reception.

2.1.2 Implemented Line Coding Schemes

The following line coding algorithms were implemented:

- Non-Return-to-Zero Level (NRZ-L)
- Non-Return-to-Zero Inverted (NRZ-I)

- Manchester Encoding
- Alternate Mark Inversion (AMI)

Each scheme operates on a user-defined binary sequence and generates a corresponding digital waveform using a configurable number of samples per bit for accurate visualization.

2.1.3 NRZ-L Encoding

In NRZ-L, each bit is represented by a constant voltage level over the entire bit duration, where `1` maps to a positive level and `0` to a negative level. While bandwidth-efficient and simple, long runs of identical bits may cause synchronization issues due to the lack of transitions.

2.1.4 NRZ-I Encoding

NRZ-I encodes data using transitions rather than fixed voltage levels. A `1` causes a voltage inversion at the beginning of the bit interval, while a `0` causes no transition. This improves synchronization compared to NRZ-L, as transitions depend on the transmitted data.

2.1.5 Manchester Encoding

Manchester encoding represents each bit with a mid-bit transition: `1` is encoded as high-to-low and `0` as low-to-high. This guarantees reliable clock recovery at the cost of increased bandwidth consumption.

2.1.6 AMI Encoding

In AMI, `0` is represented by zero voltage, while successive `1`s alternate between positive and negative voltage levels. This eliminates the DC component and improves signal reliability, though long sequences of zeros still lack transitions.

2.1.7 Implementation Structure

Each line coding scheme was implemented as a separate function accepting a binary input sequence and the number of samples per bit. The output is a discrete-time encoded signal, visualized using a unified plotting routine for direct comparison. Input validation ensures that only valid binary data is processed, enhancing robustness.

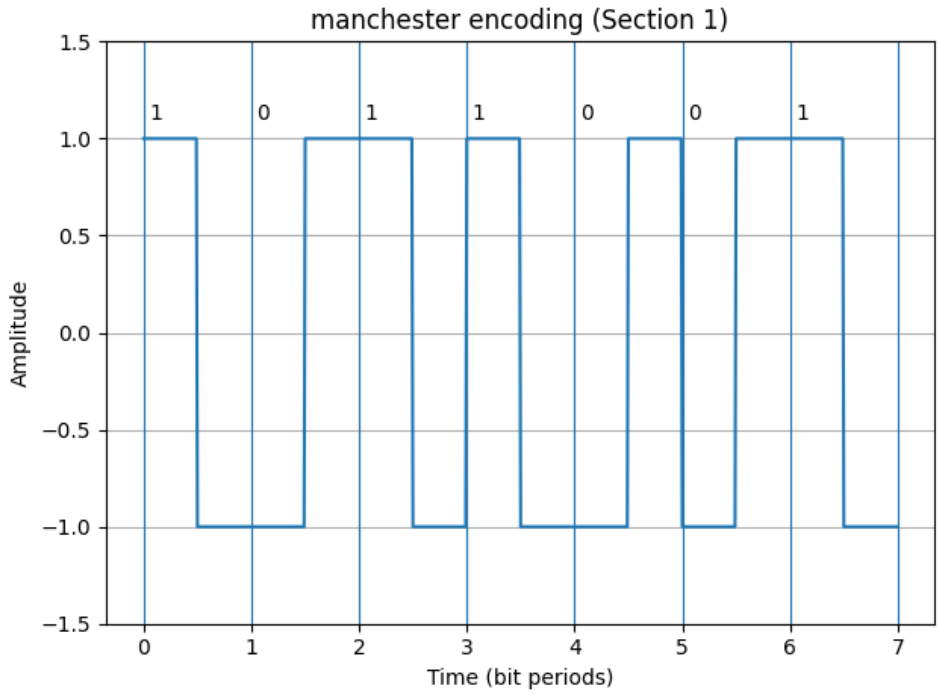

Figure 1: Digital-to-digital conversion example using Manchester line coding for the bitstream 1011001.

2.2 Digital-to-Analog Conversion

2.2.1 Overview and Objective

Digital-to-analog conversion transmits digital information by modulating a continuous-time carrier signal. Instead of direct voltage levels, binary data is encoded by varying the carrier's amplitude, frequency, or phase. The objective of this section is to implement common digital modulation schemes and verify correct data recovery through corresponding demodulation.

2.2.2 Implemented Modulation Schemes

The following digital modulation schemes were implemented:

- Amplitude Shift Keying (ASK / On-Off Keying)
- Frequency Shift Keying (FSK)
- Binary Phase Shift Keying (BPSK)

Each scheme converts a binary input sequence into an analog waveform, which is then demodulated at the receiver to recover the original data.

2.2.3 Amplitude Shift Keying (ASK)

In ASK modulation, the carrier amplitude depends on the transmitted bit: `1` enables the carrier, while `0` suppresses it. On-Off Keying (OOK), used in this project, is simple to implement but sensitive to noise due to its reliance on amplitude variations.

2.2.4 Frequency Shift Keying (FSK)

FSK represents binary symbols using different carrier frequencies, $f_0 = f_c - \Delta f$ and $f_1 = f_c + \Delta f$. This separation ensures reliable detection and provides better robustness to amplitude noise compared to ASK.

2.2.5 Binary Phase Shift Keying (BPSK)

In BPSK, information is encoded in the carrier phase: `1` corresponds to phase 0 and `0` to phase π. This scheme offers strong noise immunity and is widely used in practical digital communication systems.

2.2.6 Implementation Structure

All modulation schemes were implemented within a unified framework. The input bitstream is divided into fixed-duration intervals, and a sinusoidal carrier is generated with predefined parameters. Depending on the selected scheme, the carrier's amplitude, frequency, or phase is modified accordingly. Demodulation is performed using coherent detection and simple decision rules, with shared parameters between the modulator and demodulator to ensure consistency.

2.2.7 Verification and Output

For each modulation scheme, the modulated analog waveform and the recovered digital bit sequence are generated. Correct operation is verified by comparing the recovered bits with the original input and by visual inspection of the modulated signals.

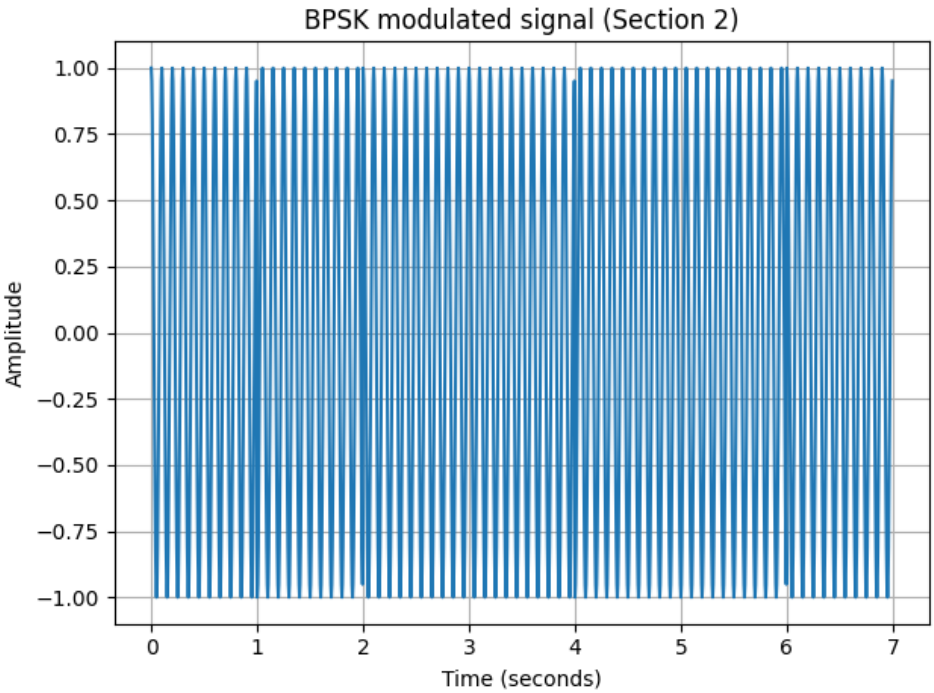

Figure 2: Digital-to-analog conversion example using BPSK modulation for the bitstream 1011001 (transmitted modulated waveform).

2.3 Analog-to-Digital Conversion

2.3.1 Overview and Objective

Analog-to-digital conversion transforms a continuous-time signal into a digital representation suitable for processing and transmission. In this project, Pulse Code Modulation (PCM) is implemented, consisting of sampling, quantization, and binary encoding. The objective is to realize a complete PCM chain and verify correct digital representation and reconstruction of an analog signal.

2.3.2 PCM Processing Stages

The implemented PCM system includes the following stages:

- Sampling
- Uniform quantization
- Binary encoding
- Decoding and signal reconstruction

Each stage is implemented explicitly to observe its impact on the signal.

2.3.3 Analog Signal Model

The input signal is modeled as a sinusoidal waveform:

$$x(t) = \sin(2\pi f_m t)$$

where f_m is the message frequency. A high-resolution time axis is used for reference and visualization.

2.3.4 Sampling

Sampling is performed at uniform intervals with sampling period $T_s = 1/f_s$. The analog signal is evaluated at discrete sampling instants. A warning is issued if the Nyquist condition ($f_s < 2f_m$) is violated to indicate potential aliasing.

2.3.5 Quantization

Uniform quantization is applied with a user-defined number of bits per sample. The number of quantization levels is $L = 2^b$, where b is the quantization bit depth. Each sample is mapped to the nearest level within a fixed amplitude range, introducing quantization error.

2.3.6 PCM Encoding, Decoding, and Reconstruction

Quantized sample indices are encoded into fixed-length binary words to form the PCM bitstream. At the receiver, the bitstream is decoded back into indices, which are then mapped to their corresponding amplitude levels using a mid-rise reconstruction approach. The reconstructed signal is compared with the original signal to assess distortion due to sampling and quantization.

2.3.7 Implementation Structure and Verification

The system is implemented modularly, with separate functions for signal generation, sampling, quantization, and PCM encoding/decoding. The sampled, quantized, and reconstructed signals are visualized to illustrate each stage. Correct operation is verified by consistency between encoded and decoded data and by visual inspection of the reconstructed signal.

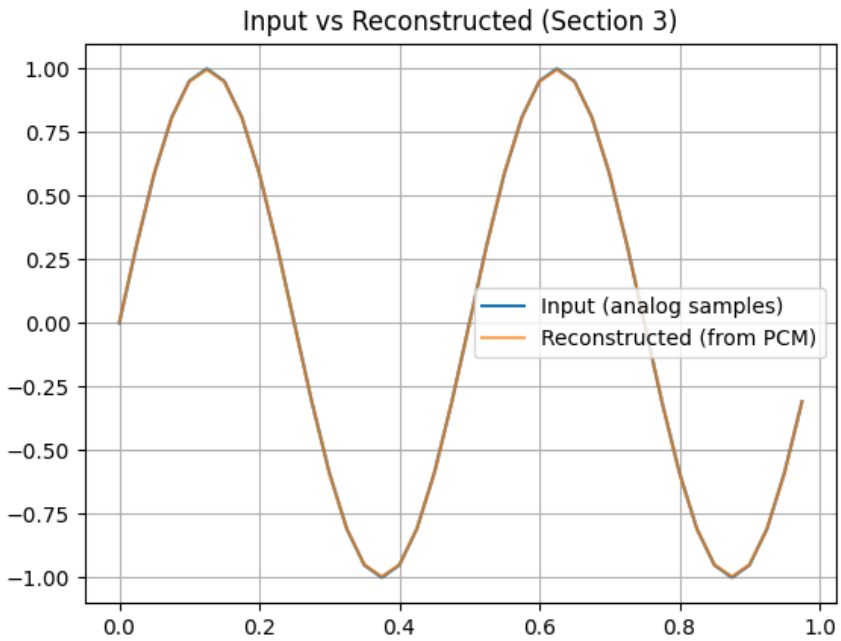

Figure 3: Analog-to-digital conversion (PCM): original message $m(t) = \sin(2\pi f_m t)$ overlaid with the reconstructed signal after sampling ($f_s = 40$ Hz) and quantization (8 bits/sample), using $f_m = 2$ Hz.

2.4 Analog-to-Analog Conversion

2.4.1 Overview and Objective

Analog-to-analog conversion transmits an analog message by modulating a higher-frequency carrier, enabling efficient use of band-limited channels. The objective of this section is to implement common analog modulation techniques and verify correct recovery of the message signal through appropriate demodulation.

2.4.2 Implemented Modulation Schemes

The following analog modulation schemes were implemented:

- Amplitude Modulation (AM)
- Phase Modulation (PM)
- Frequency Modulation (FM)

Each scheme modulates a sinusoidal carrier using a low-frequency message signal and applies a corresponding demodulation method.

2.4.3 Message and Carrier Signals

The message signal is modeled as

$$m(t) = \sin(2\pi f_m t),$$

and the carrier signal as

$$c(t) = \cos(2\pi f_c t),$$

with $f_c \gg f_m$ to ensure proper modulation. A sufficiently high internal sampling rate is used for accurate processing and visualization.

2.4.4 Amplitude Modulation (AM)

In AM, the carrier amplitude is varied proportionally to the message signal:

$$s(t) = (1 + k_a m(t)) \cos(2\pi f_c t).$$

Demodulation is performed using envelope detection followed by low-pass filtering to recover the message signal.

2.4.5 Phase Modulation (PM)

PM varies the carrier phase according to the message signal:

$$s(t) = \cos\left(2\pi f_c t + k_p m(t)\right).$$

Demodulation is achieved by extracting and unwrapping the instantaneous phase using I/Q components, which is then scaled to obtain the message.

2.4.6 Frequency Modulation (FM)

In FM, the carrier's instantaneous frequency varies with the message:

$$s(t) = \cos\left(2\pi f_c t + k_f \int m(t)\, dt\right).$$

Demodulation is implemented by differentiating the unwrapped instantaneous phase and applying low-pass filtering to suppress noise.

2.4.7 Implementation Structure and Verification

The system is implemented modularly, with separate functions for each modulation and demodulation scheme and shared utilities for phase extraction and filtering. For each technique, the original message, modulated carrier, and recovered message are visualized.

Correct operation is verified by comparing the recovered and original signals, with minor boundary deviations attributed to filtering effects.

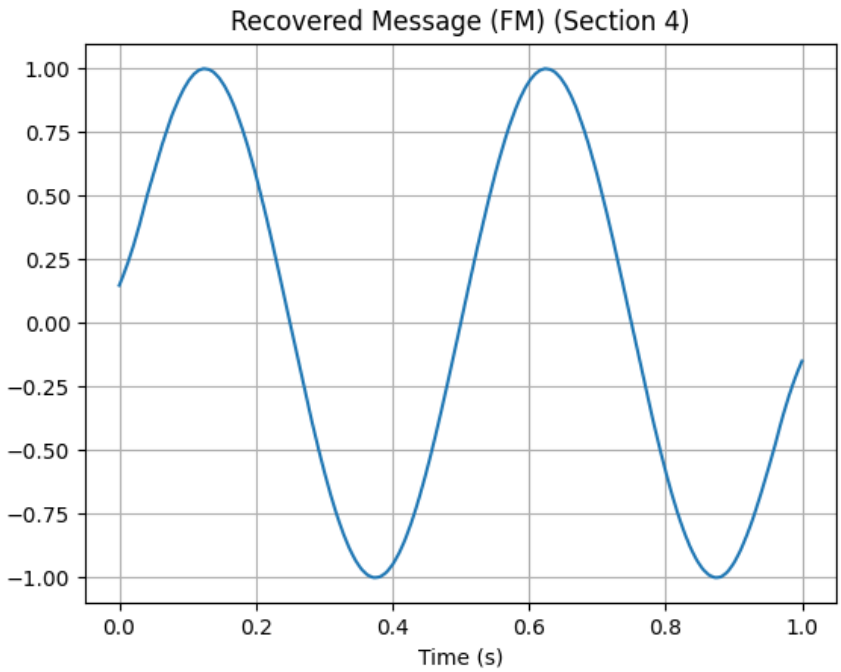

Figure 4: Analog-to-analog conversion example using FM: original message signal and recovered message after FM demodulation with low-pass filtering ($f_m = 2$ Hz, $f_c = 60$ Hz).

2.5 Web Interface

In order to represent our results, we implemented simple webapp that is implemented on top of CLI socket programming. Client A(webapp) sends encodes and sends the message to server B (terminal). To make it easier, we made visual graph under sender's part. User selects communication methods out of d2d, d2a, a2d and a2a, then for each of them, user can select encoding/modulation tecnique from drop down list. Figure below shows examples for BPSK and Manchester modulation and encoding technicques. Besides, textboxes contain default values, user can chnage them to see how the encoded/modulated signal chnages with changing the parameter.

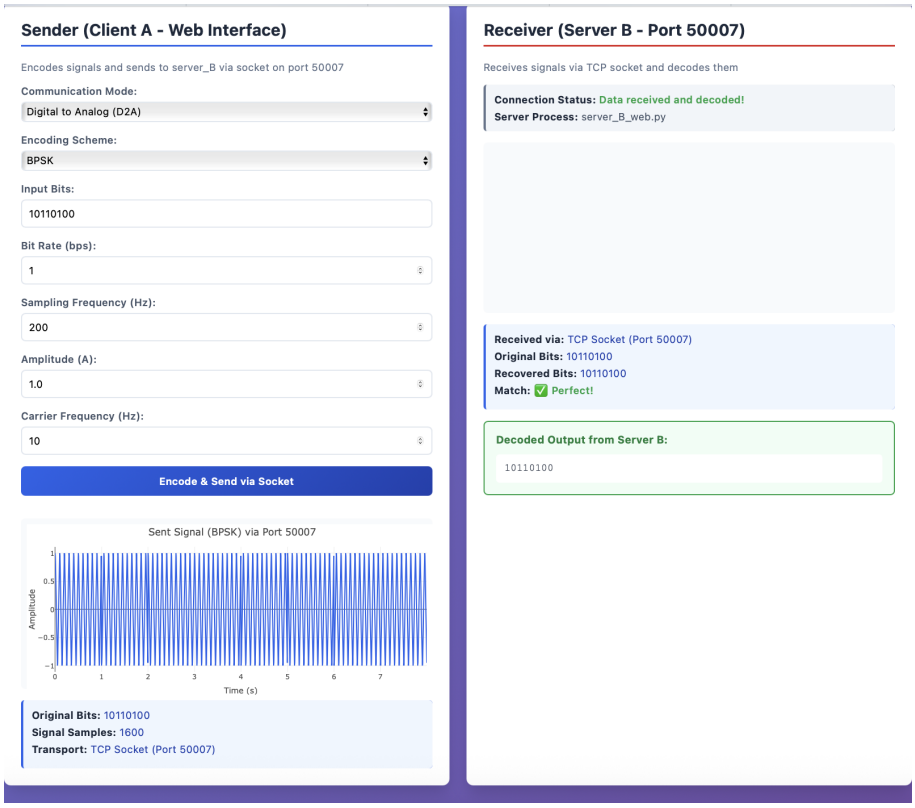

Figure 5: Webapp screenshot for BPSK

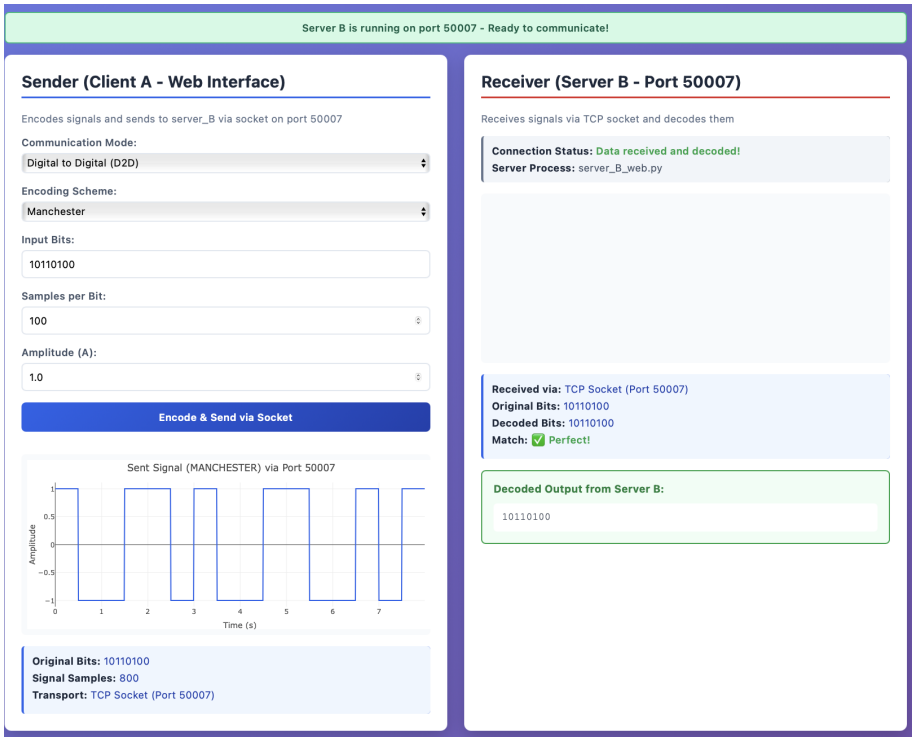

Figure 6: Webapp screenshot for Manchester

3 AI OPTIMIZATION and BENCHMARK ANALYSIS

We gave our code for all 4 cases to Gemini 3 Pro and Claude Sonnet 4.5 models. Both models had the same prompt: "go over my codec code in files named digital2digital.py, digital2analog.py, analog2digital.py, analog2analog.py and then optimize them in terms of speed and space. create a new files called a2a_gemini.py, d2d_gemini.py, a2d_gemini.py, d2c_gemini.py for corresponding files.". Files from Claude were saved in "..._claude.py" files.

3.1 Claude's Optimization

Firstly, we append each bit of transmitted bit string one by one into the empty-initialized list. The appending creates a memory space repeatedly for each bit. Then we concatenate all elements in the list, which creates a new array and copies the content. This is very simple and straightforward implementation, however is not suitable for real communication due to memory fragmentation risk for long bit strings.
Claude allocated memory for array from the beginning, since we know the size of incoming data. It uses array structure and indexing, which is very fast. Below we provide the code for both our and Claude's implementation.

```
x_parts = [] # Empty list

elif scheme ==  nrz-i :
# Start from an initial level, then flip when you see a '1'
level = float(initial_level)
for b in bits:
    if b == 1:
        level = -level            # '1' means invert at start of bit
    x_parts.append(np.full(spb, level, dtype=float))

x = np.concatenate(x_parts) if x_parts else np.array([], dtype=float) #
   Concatenate copies all data
```

Listing 1: Our code fragment

```
 total_samples = n_bits * spb

    # Pre-allocate output array (faster than concatenation)
    x = np.zeros(total_samples, dtype=float)
  elif scheme ==  nrz-i :
        # Track level changes
        level = float(initial_level)
```

```
        for i, b in enumerate(bits):
            if b == 1:
                level = -level
            x[i*spb:(i+1)*spb] = level
```

Listing 2: Claude's code fragment

Second improvement is vectorization instead of processing the bits one by one in the loop. Bit by bit processing cannot use CPU parallelism, while vectors can be processed all at once. This is SIMD(single instruction multiple data) instruction, which can be parallelized in CPU.

```
for b in bits:
    level = +A if b == 1 else -A
    x_parts.append(np.full(spb, level, dtype=float))
```

Listing 3: Our code fragment

```
# Vectorized: directly map bits to levels and repeat
levels = np.where(bits == 1, A, -A)
x = np.repeat(levels, spb)
```

Listing 4: Claude's code fragment

In Manchester encoding, our code creates two arrays, concatenates them and appends to list. In optimized version, direct slice assignments to pre-allocated array is used. Same direct accessing is used in AMI encoding.

```
for b in bits:
    # 1 -> high then low, 0 -> low then high
    if b == 1:
        first, second = +A, -A
    else:
        first, second = -A, +A

    x_parts.append(np.concatenate([
        np.full(half, first, dtype=float),
        np.full(half, second, dtype=float)
    ]))
```

Listing 5: Our code fragment Manchester encoding

```
# Vectorized Manchester encoding
for i, b in enumerate(bits):
    if b == 1:
        x[i*spb:i*spb+half] = A
        x[i*spb+half:(i+1)*spb] = -A
```

```
    else:
        x[i*spb:i*spb+half] = -A
        x[i*spb+half:(i+1)*spb] = A
```

Listing 6: Claude's code fragment Manchester encoding

For NRZ-L decoding, all bits are processed in a loop. This is improved by reshaping 1D input to 2D array and finding mean and comparing in constant time.

```
if scheme == nrz-l :
    # Decide each bit by the sign of the average level in its segment
    for i in range(n_bits):
        seg = x[i*spb:(i+1)*spb]
        avg = float(np.mean(seg))
        bits_out.append(1 if avg >= 0 else 0)
```

Listing 7: Our code fragment

```
# Reshape signal into (n_bits, samples_per_bit) for batch processing
x_reshaped = x.reshape(n_bits, spb)

if scheme == nrz-l :
    # Vectorized: compute mean of each bit segment and threshold
    avg = np.mean(x_reshaped, axis=1)
    bits_out = (avg >= 0).astype(int).tolist()
```

Listing 8: Claude's code fragment

Similarly, Manchester decoding uses batch vectorization. It uses two array slices compute all first/second half averages and the boolean operation replace loop conditionals. Batched vectorization is used in AMI decoding as well.

```
for i in range(n_bits):
    seg = x[i*spb:(i+1)*spb]
    first_avg = float(np.mean(seg[:half]))
    second_avg = float(np.mean(seg[half:]))

    if first_avg >= 0 and second_avg < 0:
        bits_out.append(1)
    elif first_avg < 0 and second_avg >= 0:
        bits_out.append(0)
    else:
        raise ValueError( ... )
```

Listing 9: Our code fragment

```
# Vectorized: compute first and second half averages
first_half = np.mean(x_reshaped[:, :half], axis=1)
```

```
second_half = np.mean(x_reshaped[:, half:], axis=1)

# 1: + then -, 0: - then +
bits_out = ((first_half >= 0) & (second_half < 0)).astype(int).tolist()
```

Listing 10: Claude's code fragment

3.2 Gemini's Optimization

In digital2analog.py, we process the message bit by bit. Gemini optimizes this by using vectorization similar to Claude's optimization. It expands bits for the whole duration instantly and uses vector operation instead of looping. This is O(n) to O(1) optimization.

```
# Iterates 1000 times for 1000 bits
for i, b in enumerate(bits):
    idx0 = i * spb
    idx1 = (i + 1) * spb
    if b == 1:
        s[idx0:idx1] = A * np.cos(2 * np.pi * fc * t[idx0:idx1])
```

Listing 11: Our code fragment:

```
bits_expanded = np.repeat(bits, N)
amplitude = np.where(bits_expanded == 1, A, 0.0)
s = amplitude * np.cos(2 * np.pi * fc * t)
```

Listing 12: Gemini's code fragment: Vectorized Modulation

Similar to Claude's optimization, mean value is found using numpy.mean and reshaping into 2D array. This replaces iteration over each bit.
In our implementation, we get inputs as strings, then parse them as integers. Gemini uses integers rightaway. Getting rid of type conversion and use of integers speeds up process.

```
# Converts integer to binary string (e.g. 5 ->  101 )
for q in q_indices:
    b = format(int(q), f'0{bits}b')
    bitstream.extend([int(bit) for bit in b]) # Loop over string chars
```

Listing 13: Our code fragment

```
# Uses bitwise AND mask [4, 2, 1] to extract bits purely with math
powers = 1 << np.arange(bits - 1, -1, -1)
bit_matrix = (q_indices[:, None] & powers) > 0
return bit_matrix.astype(int).flatten().tolist()
```

Listing 14: Gemini's code fragment: Bitwise Math Encoding (No Strings)

Gemini uses linear algebra (dot product) to convert bits back to integers instead of parsing strings.

```
# Reconstructs string from bits, then parses string to int
for i in range(0, len(bitstream), bits):
    b = bitstream[i:i+bits]
    q = int(  .join(str(bit) for bit in b), 2)
```

Listing 15: Our code fragment

```
# Dot product of bits [1, 0, 1] with powers [4, 2, 1] -> 5
bit_matrix = bit_array[:n_samples*bits].reshape(n_samples, bits)
powers = 1 << np.arange(bits - 1, -1, -1)
q_indices = bit_matrix.dot(powers)
```

Listing 16: Gemini's code fragment: Dot Product Decoding

Gemini uses cumulative product to calculate state dependencies (where the next value depends on the previous one) without a loop.

```
# Loop required because 'level' depends on the previous iteration
for b in bits:
    if b == 1:
        level = -level
    x_parts.append(np.full(spb, level, dtype=float))
```

Listing 17: Our code fragment

```
#  Cumprod  tracks the flipping state in a single vectorized step
changes = np.where(bits == 1, -1.0, 1.0)
states = initial_level * np.cumprod(changes)
x = np.repeat(states, spb)
```

Listing 18: Gemini's code fragment: Cumprod for State Tracking (NRZ-I)

3.3 Benchmark analysis

Given AI optimizations improve both time complexity and space complexity. To compare AI's results, we did benchmark experiments for each of modulation/encoding tecnique. Encoding experiments have random, alternating, all-ones and all-zeros inputs. Modulation techniques have 500, 1000, and 2000 samples as test inputs.

In general, Gemini's version performs well in encoding techniques, reaching speed up up to 44 times. On the other hand, Claude's version performs well on analog-to-digital and ASK modulation methods. In D2D code, The Gemini code utilizes NumPy vectorization to calculate these state transitions in a single pass which makes it fast.

For analog-to-digital modulations, Claude had better scaling. Despite being worse for 500 samples, it is almost twice as fast for 2000 samples. Claude's implementation has lower algorithmic complexity (O(N)), making it superior for larger data.

We logged results of benchmark experiments into benchmark_results.csv. This bar chart summarized memory and time complexities of our, Gemini's and Claude's implementations. These bar charts (Figure 7) essentially show time-space tradeoff. Our version is more memory-efficient compared to AI-speed-optimized versions. However, given that the increase is only about 700 KB, the 3x-10x speedup is generally considered a successful optimization.

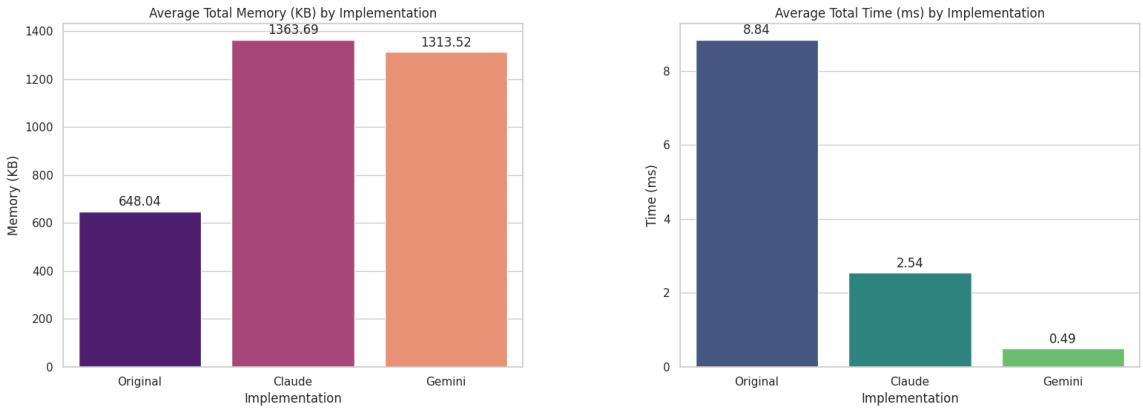

Figure 7: Comparison of Memory Usage and Time Execution

4 DISCUSSION

This section discusses and compares the implemented communication techniques across all four conversion categories. The discussion focuses on signal behavior, system complexity, bandwidth requirements, and robustness to distortions, based on the observed results and generated waveforms.

4.1 Digital-to-Digital Conversion

The digital-to-digital conversion schemes demonstrate clear differences in transition density and signal structure. NRZ-L provides the simplest representation but suffers from long sequences without transitions, which may cause synchronization issues. NRZ-I improves synchronization by encoding information in transitions rather than absolute levels.

Manchester encoding guarantees a transition at every bit interval, resulting in excellent clock recovery performance. However, this comes at the expense of increased bandwidth

usage. AMI successfully eliminates the DC component by alternating pulse polarity for logical ones, which is advantageous for transformer-coupled channels, although long runs of zeros still lack transitions.

These behaviors are clearly visible in the generated digital waveforms and are consistent with theoretical expectations.

4.2 Digital-to-Analog Conversion

In the digital-to-analog conversion stage, ASK, FSK, and BPSK modulation schemes were implemented and evaluated. ASK exhibits the simplest structure but is highly sensitive to amplitude noise. FSK improves robustness by encoding information in frequency variations, making it less vulnerable to amplitude distortions.

BPSK offers the highest noise immunity among the implemented schemes, as information is encoded solely in the carrier phase. The demodulated results confirm that coherent detection allows reliable bit recovery when modulation parameters are properly chosen.

4.3 Analog-to-Digital Conversion

The analog-to-digital conversion using PCM highlights the impact of sampling frequency and quantization resolution on signal quality. Sampling below the Nyquist rate introduces aliasing effects, while increasing the sampling frequency improves signal representation.

Quantization introduces a finite resolution error that becomes more apparent at lower bit depths. Increasing the number of quantization bits significantly reduces distortion in the reconstructed signal, at the cost of a higher bit rate. These trade-offs are clearly observable in the sampled, quantized, and reconstructed signal plots.

4.4 Analog-to-Analog Conversion

The analog-to-analog modulation schemes demonstrate distinct characteristics in terms of signal behavior and demodulation complexity. AM modulation allows simple envelope detection but is sensitive to amplitude variations and requires proper carrier-to-message frequency separation.

PM and FM encode information in phase and frequency variations, respectively, making them more resistant to amplitude noise. FM demodulation, however, requires phase differentiation, which amplifies high-frequency noise and necessitates low-pass filtering for clean signal recovery. The observed demodulated waveforms confirm the importance of proper filtering in FM systems.

4.5 Overall Comparison

Across all conversion types, a clear trade-off emerges between implementation complexity, bandwidth efficiency, and robustness to noise. Simpler schemes tend to be easier to implement but are more sensitive to distortions, while more advanced schemes offer improved performance at the cost of increased complexity. The results obtained in this project align well with established communication theory.

4.6 Difficulties encountered

During the implementation of the communication system, several practical issues were observed across the different conversion stages. In digital line coding, it was found that schemes such as NRZ-I and Manchester encoding can follow different valid conventions, which made it important to maintain consistency between the encoder and decoder to ensure correct data recovery.
In the digital-to-analog stage, modulation techniques like BPSK encode information only in the phase of the carrier, which makes visual inspection of the waveform less straightforward and requires correctness to be verified through demodulation results. For analog-to-digital conversion using PCM, a noticeable trade-off was observed between signal quality and data rate, as higher sampling frequencies and increased quantization resolution improve reconstruction accuracy but also lead to larger bitstreams. In analog-to-analog conversion, especially in frequency modulation, the demodulation process was sensitive to noise due to phase differentiation, making low-pass filtering necessary to obtain a clean recovered signal. Additionally, small distortions near the beginning and end of signals were observed as a result of filtering edge effects, which are common in practical signal processing implementations.

5 Conclusion

In this project, a comprehensive communication system covering all major signal conversion categories was designed and implemented. Digital-to-digital, digital-to-analog, analog-to-digital, and analog-to-analog conversion techniques were studied and realized using modular and extensible software implementations.

Digital-to-digital conversion was implemented using multiple line coding schemes, demonstrating how binary information can be represented using different voltage-level and transition-based techniques. Digital-to-analog conversion illustrated how digital data can be transmitted over analog channels using carrier modulation, highlighting the trade-

offs between simplicity, bandwidth efficiency, and noise robustness.

Analog-to-digital conversion was implemented using a complete Pulse Code Modulation (PCM) chain, including sampling, quantization, binary encoding, and reconstruction. The results clearly showed the impact of sampling frequency and quantization resolution on signal quality, as well as the fundamental trade-off between signal fidelity and bit rate.

Finally, analog-to-analog conversion was realized using amplitude, phase, and frequency modulation. The implemented demodulation techniques successfully recovered the original message signals, demonstrating the importance of proper carrier selection and filtering, particularly in frequency modulation systems.

Overall, the project provides a unified and practical view of core communication system concepts. The implemented simulations confirm theoretical expectations and illustrate the relationships between signal representation, system complexity, and performance. This work establishes a solid foundation for understanding more advanced digital and analog communication techniques.

REFERENCES